گفتگو و گزارش اجرای پروژه

سامانهٔ بازرسی تصویری ترک تیرهای بتن‌آرمه — CrackLens

این سند، شرح اولیهٔ پروژه، روند انجام کار، وضعیت فنی خروجی و پرسش‌وپاسخ‌های کاربر دربارهٔ استفاده، نتیجهٔ Verdict، اشتراک‌گذاری برنامه و تصویر تست را برای ارائه و اشتراک در یک فایل گردآوری می‌کند.

تاریخ تهیه: ۱۴۰۵/۰۷/۲۴



تصویر نمونهٔ تیر ترک‌خوردهٔ تولیدشده برای آزمون نمایشی برنامه؛ تصویر فاقد مقیاس کالیبره است.

Project	CrackLens
Contents	شرح پروژه، اجرا، نتایج و گفتگوهای اصلی

۱. موضوع پروژه و درخواست اولیه

در فایل پیوست‌شده، ساخت خودکار پروژهٔ CrackLens از صفر تا تحویل درخواست شده بود: سامانه‌ای برای قطعه‌بندی ترک در تصویر تیر بتن‌آرمه، اندازه‌گیری طول و عرض با مقیاس مرجع، دسته‌بندی تقریبی ترک خمشی و برشی از روی زاویه، مقایسهٔ عرض با حدود سرویس‌پذیری ACI 224R و مبحث ۹ مقررات ملی ساختمان، ارائهٔ رابط Gradio و گزارش بازتولیدپذیر.

در پیوست پایانی، متن فایل اولیهٔ پروژه آمده است. در ادامه، نتیجهٔ اجرای واقعی در محیط موجود و حدود آن ثبت شده تا تفاوت میان اجزای پیاده‌شده و مواردی که به‌علت محدودیت محیط کامل نشدند روشن باشد.

۲. وضعیت اجرای پروژه

ساختار پروژه و اجزای اصلی پیاده‌سازی شدند: آماده‌سازی داده، مدل پایهٔ آستانه‌ای، محاسبهٔ هندسهٔ ترک، دسته‌بندی جهت، غربالگری کد، رابط Gradio، فرمان‌های خط فرمان، تست‌ها و نوت‌بوک گزارش.

داده: 24 تصویر مصنوعی جفت‌شده با ماسک تولید شد. دانلود دو منبع عمومی به‌علت محدودیت دسترسی شبکه در محیط اجرا ناموفق بود؛ در نتیجه، نتیجه‌های زیر مربوط به آزمون نرم‌افزار روی دادهٔ مصنوعی هستند، نه ارزیابی تعمیم به عکس‌های واقعی.

مدل اجراشده: مدل پایهٔ کلاسیک با آستانهٔ روشنایی 40؛ آموزش YOLO انجام نشد. در این مدل، mAP واقعی تعریف نشده و به‌درستی به‌صورت unavailable ثبت شده است.

Split	Images	IoU	F1
train	17	0.9973	0.9986
val	3	0.9975	0.9987
test	4	0.9955	0.9977

اعتبارسنجی: کامپایل پایتون انجام شد و ۷ تست واحد با unittest موفق بودند. نوت‌بوک گزارش اجرا شد. رابط Gradio در همین محیط اجرا نشد، چون بستهٔ Gradio نصب نبود و نصب آنلاین آن به‌علت محدودیت شبکه ممکن نشد؛ کد رابط در پروژه موجود است.

محدودیت اندازه‌گیری: اگر مرجع پیکسلی وارد نشود، فرض پیش‌فرض ۱۰۰ میلی‌متر در عرض تصویر به‌کار می‌رود. برای عکس واقعی باید مقیاس با خط‌کش یا نشانگر کالیبره شود. مقدار پیش‌فرض مبحث ۹ برابر ۰٫۳۰ میلی‌متر یک مقدار قابل‌تنظیم پروژه‌ای است و باید با نسخه و ضوابط موردنظر تطبیق داده شود. نوار CAUTION نیز قرارداد هشدار برنامه است، نه الزام مستقل کد.

فایل‌های اصلی پروژه: models/cracklens_classical.json، ASSUMPTIONS.md، README.md، notebooks. و پوشه‌های src، tests و models/metrics.json

۳. روند انجام کار

• پیوست کاربر به‌عنوان مشخصات کامل پروژهٔ CrackLens خوانده شد و اجرای انتهابه‌انتها در محدودهٔ امکان محیط آغاز شد.

• مخزن در آغاز خالی بود. به‌علت مسدودبودن دسترسی اینترنت، پروژه به مسیر آفلاین مجهز شد و دادهٔ مصنوعیِ قابل‌بازتولید به‌عنوان fallback ثبت شد.

• ماژول‌های قطعه‌بندی پایه، طول و عرض، زاویه و دسته‌بندی، verdict، رابط کاربری، خط فرمان، تست‌ها و گزارش ساخته شدند.

• دادهٔ نمونه ساخته و مدل آستانه‌ای آموزش داده شد؛ چهار تصویر آزمون تحلیل شدند و تصویرهای overlay و گزارش JSON تولید شدند.

• در یک دور اصلاح، ورودی دادهٔ جفت‌شده از ساختارهای محلی رایج اضافه شد و mAP برای مدل آستانه‌ای به‌جای عدد ساختگی، unavailable ثبت شد.

• در پایان، هفت تست موفق شدند و نوت‌بوک گزارش اجرا شد. نصب Gradio و pytest در محیط محدود باقی ماند.

۴. گفتگوی کاربر و پاسخ‌ها

درخواست کاربر

به‌عنوان کاربر چطور می‌توانم از برنامه استفاده کنم و خروجی بگیرم؟

پاسخ دستیار

برای استفاده در PowerShell، ترمینال را در پوشهٔ پروژه D:\ai-amirali باز کنید. اگر وابستگی‌ها نصب نیستند، یک‌بار محیط مجازی را بسازید و بسته‌ها را نصب کنید:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

سپس رابط برنامه را اجرا کنید:

```
$env:PYTHONPATH = "src"
python -m cracklens demo
```

در صفحه‌ای که برنامه باز می‌کند، تصویر تیر را بارگذاری کنید، طول مرجع معلوم را برحسب میلی‌متر و پیکسل وارد کنید، کد و شرایط محیطی را انتخاب کنید و Analyze cracks را بزنید. جدول شامل نوع، زاویه، طول، بیشینه و میانگین عرض و verdict است.

برای ذخیرهٔ JSON و تصویر overlay از فرمان analyze استفاده کنید. اگر مرجع معلوم ندارید، مقدار پیش‌فرض کالیبره نیست و اندازه‌ها را واقعی تلقی نکنید.

درخواست کاربر

ستون verdict نشان‌دهندهٔ چه نتیجه‌ای را برای من مشخص می‌کند؟

پاسخ دستیار

ستون Verdict نتیجهٔ غربالگری عرض ترک را نسبت به حدی که در برنامه انتخاب کرده‌اید نشان می‌دهد: PASS یعنی عرض تا ۸۰٪ حد انتخاب‌شده است؛ CAUTION یعنی عرض بین ۸۰٪ حد و خود حد است؛ FAIL یعنی عرض از حد انتخاب‌شده بیشتر است. برای حد ۰٫۳۰ میلی‌متر، تا ۰٫۲۴ PASS، از ۰٫۲۴ تا ۰٫۳۰ CAUTION و بالاتر از ۰٫۳۰ FAIL می‌شود. مرز ۸۰٪ قرارداد داخلی برنامه است و این برچسب‌ها تأیید ایمنی سازه نیستند؛ دقتشان به کالیبراسیون مقیاس تصویر وابسته است.

درخواست کاربر

در یک پاراگراف به زبان ساده کارکرد برنامه را برای من شرح بده.

پاسخ دستیار

برنامه تصویر تیر بتنی را بررسی می‌کند، ترک‌های قابل‌مشاهده را روی تصویر مشخص می‌کند و برای هر ترک طول، عرض و زاویه‌اش را تخمین می‌زند؛ سپس بر اساس زاویه، ترک را خمشی یا برشی دسته‌بندی می‌کند و عرضش را با حد انتخاب‌شده مقایسه می‌کند تا نتیجه‌ای مثل PASS، CAUTION یا FAIL نشان دهد. برای اینکه اندازه‌ها به میلی‌متر نزدیک باشند، باید مقیاس تصویر را با یک خط‌کش یا مرجع معلوم تنظیم کنید.

درخواست کاربر

برای اینکه بتوانم برنامهٔ قابل‌اجرا را برای یک نفر بفرستم، چه فایل‌هایی را نیاز است برای وی بفرستم و آن شخص چه کارهایی را باید انجام دهد؟

پاسخ دستیار

پروژه فعلاً فایل اجرایی مستقل EXE نیست؛ ساده‌ترین راه فرستادن ZIP از پوشهٔ پروژه است. برای اجرای برنامه لازم است پوشهٔ src، مدل models/cracklens_classical.json، فایل‌های requirements.txt و README.md و نیز configs، pyproject.toml و ASSUMPTIONS.md داخل ZIP باشند. داده‌های آموزشی، نوت‌بوک‌ها و تست‌ها برای اجرای فقط رابط ضروری نیستند؛ metrics.json و یک تصویر نمونه اختیاری‌اند.

گیرنده باید Python نسخهٔ ۳٫۱۰ یا بالاتر نصب کند، ZIP را باز کند، در PowerShell وارد پوشهٔ پروژه شود و این فرمان‌ها را اجرا کند:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
$env:PYTHONPATH = "src"
python -m cracklens demo
```

بعد از بارگذاری تصویر و تنظیم کالیبراسیون، برنامه را اجرا می‌کند. برای ذخیرهٔ خروجی JSON و PNG، فرمان python -m cracklens analyze را با مسیر تصویر و پارامترهای مرجع اجرا می‌کند. نصب وابستگی‌ها معمولاً اینترنت می‌خواهد. مدل فعلی با دادهٔ مصنوعی آموزش دیده و برای تست اجراست، نه تصمیم مهندسی دربارهٔ تصاویر واقعی.

درخواست کاربر

یک نمونه تصویر تیر برای تست برنامه به من بده.

پاسخ دستیار

تصویر نمونهٔ تیر ترک‌خورده ساخته و در پیام گفتگو ارائه شد. نسخهٔ محلی آن نیز در پوشهٔ artifacts با نام beam_test.png ذخیره شد تا داخل برنامه بارگذاری شود. تصویر خط‌کش کالیبراسیون ندارد؛ برای تست ظاهر خروجی مناسب است، اما اندازه‌های عرض ترک را واقعی تلقی نکنید.

۵. نتیجهٔ تحلیل نمونه و شیوهٔ خروجی‌گرفتن

برای خروجی فایل، فرمان زیر یک فایل JSON و یک تصویر overlay با همان نام در پوشهٔ artifacts می‌سازد. مقادیر مرجع را با اندازهٔ واقعی نشانگر داخل عکس جایگزین کنید:

```
python -m cracklens analyze `
  --image "C:\path\beam.jpg" `
  --output "artifacts\beam01.json" `
  --reference-mm 50 `
  --reference-pixels 120 `
  --code ACI224R `
  --exposure humidity
```

فایل JSON شامل خلاصه و ردیف‌های ترک است؛ تصویر PNG، ماسک‌های شناسایی‌شده را روی تصویر ورودی نشان می‌دهد. رابط Gradio فعلی نتایج را روی صفحه نمایش می‌دهد و برای ذخیرهٔ جداگانه، از دستور خط فرمان بالا استفاده می‌شود.

۶. نکات و منابع ثبت‌شده در گفتگو

حدود ACI در کد به‌عنوان راهنمای غربالگری و قابل‌تنظیم معرفی شده‌اند؛ برای کاربرد مهندسی باید نسخهٔ آیین‌نامه و شرایط محیطی پروژه بررسی شود. حد مبحث ۹ در پیاده‌سازی، مقدار پیش‌فرض قابل‌تنظیم است و ادعای حد جهانی یا قطعی ندارد.

ACI reference mentioned in the prior response: https://www.concrete.org/portals/0/files/pdf/webinars/ws_S18_Tyler.pdf
DeepCrack dataset terms mentioned in the prior response: https://github.com/yhlleo/DeepCrack/blob/master/README.md?plain=1

Appendix — Original project brief

این پیوست، متن استخراج‌شده از فایل اولیهٔ کاربر است. شکست خطوط ناشی از صفحه‌آرایی فایل مبدأ برای خوانایی دوباره‌چینی شده است.

3) Master Codex Prompt: Zero-to-Hundred Automated Build of the RC Crack Inspection Project (Option 3) 2026-10-16 Contents 1 Purpose and Target Project 2 Prompt Engineering Principles Applied 3 The Master Prompt(Copy-Paste into Codex) 4 How to Run and Customize 1 Purpose and Target Project Option 3 is the cross-course project that couples the “Applications of AI in Civil Engineering” course (Dr. Amin Moghaddas) with Reinforced Concrete 1: Computer-Vision-Based Quantification of Flexural vs. Shear Cracks in RC Beams Aligned with Code-Level Serviceability Limits. The goal of the master prompt below is to have OpenAI Codex execute the project “zero to hundred” automatically: scaffold the repository, download/prepare a public dataset, train a segmentation model, quantify crack geometry, classify flexural vs. shear cracks, compare widths against ACI 224R / Iranian National Building Code (Mabhas 9) limits, and ship a Gradio demo plus a reproducible report — with human involvement limited to running the final commands and reviewing results. 2 Prompt Engineering Principles Applied The master prompt is built directly on [OI]’s official Codex Prompting Guide (Cookbook, Feb 2026). The key evidence-based rules it encodes: 1. Autonomy and persistence first. The guide identifies autonomy/persistence, codebase exploration, tool use, and frontend quality as the most critical prompt snippets for Codex, and warns that asking for upfrontplans or preambles can cause theagent to stop abruptly before the rollout completes. The prompt therefore orders end-to-end delivery and forbids stopping at a plan. 2. Working code, not plans. Codex’s default expectation is “deliver working code, not just a plan”; if details are missing, make reasonable assumptions and complete a working version. The prompt instructs the agent to make documented assumptions instead of pausing with questions. 3) 3. Tool discipline. Prefer rg for search, dedicated tools over raw shell, apply_patch for edits, and parallel tool calls — all mirrored in the prompt’s Environment & Tools section. 4. Engineering quality. Root-cause fixes, no broad try/except swallows, no silent failures, keep builds/type-checks passing, and never revert user changes — carried into the prompt’s Engineering Standards section. 5. Frontend quality. For the Gradio/web demo, the guide warns against generic “AI slop” layouts; the prompt sets a concrete design bar. 6. Plan closure. Any plan items must end Done, Blocked (with a reason and a targeted question), or Cancelled — never left in-progress. 3 The Master Prompt(Copy-Paste into Codex) Copy everything inside the block below and feed it to Codex (CLI or cloud) as a single task: # ROLE You are an autonomous senior ML/structural-engineering engineer running in Codex CLI on the user's machine. You own this project end-to-end: from an empty repository to a trained model, quantified results, a working demo, and a reproducible report. The user is a 5th-semester Civil Engineering student at Amirkabir University of Technology; the deliverable must impress a structural-health-monitoring professor and be presentable in both "Applications of AI in Civil Engineering" and "Reinforced Concrete 1". # PROJECT Build "CrackLens": an automated RC-beam crack inspection system. Pipeline: image-> crack instance segmentation-> crack geometry quantification (length, width in mm using a scale reference)-> flexural vs. shear classification (crack angle via geometry)-> comparison of crack width against code serviceability limits (ACI 224R and Iranian National Building Code, Mabhas 9)-> interactive Gradio web demo + Jupyter report notebook. # AUTONOMY & PERSISTENCE- Work autonomously and persist until the task is fully handled end-to-end in this turn. Do not stop at analysis, partial fixes, or a plan; carry every change through implementation, verification, and a clear explanation of outcomes.- Bias to action: default to implementing with reasonable assumptions; do not end your turn with clarifying questions unless truly blocked. Record every assumption in ASSUMPTIONS.md.- Never end with only a plan. The deliverable is working code and executed results.- Before finishing, reconcile every stated intention/TODO: mark each Done, Blocked (one-sentence reason + targeted question), or Cancelled. Nothing left in-progress. # EXECUTION PLAN (execute it, don't just state it) 1. **Scaffold**: create repo structure: `data/`, `src/` (data, models, quantification,

classify, app), `notebooks/`, `tests/`, `configs/`, `reports/`. Add pyproject.toml, requirements.txt, README.md, .gitignore, ASSUMPTIONS.md. 2. **Data**: programmatically download a public crack dataset (e.g., SDNET2018 or a Mendeley/Kaggle concrete-crack segmentation dataset; fall back to any openly licensed crack-segmentation dataset reachable via direct URL or `kagglehub`). Write the download/preparation script so it is idempotent and caches locally. Split train/val/test with a fixed seed. If a segmentation-mask dataset is unavailable, generate polygon masks from available annotations or synthesize masks via classical CV (thresholding + morphological cleanup) and document this decision. 3. **Model**: fine-tune a lightweight instance-segmentation model (YOLOv8n-seg / YOLO11n-seg via Ultralytics, or Mask R-CNN fallback) in PyTorch. Train on CPU/GPU-agnostic settings (Google Colab T4-compatible). Log metrics: mask mAP@50-95, IoU. Save best weights to `models/`. 4. **Quantification**: post-process predicted masks with OpenCV: skeletonization for crack length; perpendicular-width profiling for crack width; convert pixels to millimeters using a known scale reference (e.g., a marker in the image or an assumed image width); classify each crack as flexural (near-vertical) or shear (diagonal) from its principal-axis angle, reporting the angle. 5. **Code compliance module**: a function that takes crack width (mm) and returns PASS/CAUTION/FAIL against configurable thresholds (defaults drawn from ACI 224R serviceability guidance and Mabhas 9 limits; cite exact values you hard-code in the docstring and flag them as configurable). 6. **Demo**: a polished Gradio app (upload image-> segmented overlay + per-crack table: type, length mm, max/mean width mm, code verdict). Avoid generic-looking UI: purposeful typography, one clear visual direction, CSS variables, meaningful states for processing/errors; no lazy purple-on-white defaults. 7. **Report notebook**: `notebooks/report.ipynb` executed end-to-end: dataset overview, sample predictions, metrics, error analysis, and a short Persian+English summary paragraph suitable for the professor. 8. **Tests & quality**: pytest unit tests for quantification math (synthetic crack masks with known geometry), classification angle logic, and compliance thresholds. All tests, lint, and type-checks must pass before you finish. # ENVIRONMENT & TOOLS- Python 3.10+, PyTorch, Ultralytics, OpenCV, scikit-image, Gradio, pytest. Prefer CPU-light defaults; guard GPU usage with device detection.- Prefer `rg` for search; use dedicated tools over raw shell when available; use `apply_patch` for single-file edits; parallelize independent tool calls.- No internet-independent hallucinated datasets: if a dataset URL fails, try at least two documented alternatives before declaring it Blocked. # ENGINEERING STANDARDS- Correctness, clarity, and reliability over speed. Fix root causes, not symptoms; no speculative changes or hacks.- Tight error handling: no broad try/except, no silent failures, no success-shaped fallbacks; surface errors explicitly with meaningful messages.- Keep the repo clean: small single-purpose functions, validated inputs, DRY (search for prior art before adding helpers), no unnecessary casts.- Conform to the project's own emerging conventions; keep code ASCII by default; rare, only-when-needed comments.- Never revert changes you did not make; never run destructive git commands. # DELIVERABLES (definition of done) 3)- [ ] Repo runs from a clean clone with one documented command set in README.md.- [ ] Trained weights + metrics file produced and committed/persisted.- [ ] Gradio demo launches locally and produces correct outputs on 3+ sample images.- [ ] Executed report notebook with real (not fabricated) numbers.- [ ] Tests green; ASSUMPTIONS.md and README complete; final message = concise outcome summary, not a plan. 4 How to Run and Customize Run the prompt with high reasoning effort in Codex CLI (the official guide recommends medium for interactive work and high/xhigh for the hardest autonomous tasks — a multi-hour zero-to-hundred build qualifies). Because Codex can work autonomously for hours with compaction, expect one long run rather than many short prompts; do not interleave plan requests, which the guide warns can halt a rollout early. Customization knobs: swap the dataset name in step 2 if your campus network blocks one source; change the YOLO variant for speed (n to s) if you get a Colab GPU; and adjust the compliance thresholds in the docstring to whichever code edition your professor teaches. If Codex reports Blocked on dataset access, answer with your Kaggle credentials or attach the dataset locally and say “continue” — persistence rules in the prompt will resume the same plan.